

Lecture 6

Verilog HDL

Peter Cheung
Imperial College London

URL: www.ee.imperial.ac.uk/pcheung/teaching/EE2_CAS/
E-mail: p.cheung@imperial.ac.uk

Lecture Objectives

- ◆ By the end of this lecture, you should understand:
 - The basic structure of a module specified in Verilog HDL
 - Commonly used syntax of Verilog HDL
 - Continuous vs Procedural Assignments
 - **always** block in Verilog and sensitivity list
 - Nets vs variables
 - The use of arithmetic and logic operations in Verilog
 - The danger of **incomplete specification**
 - How to specify **clocked circuits**
 - Differences between **blocking** and **nonblocking** assignments

Schematic vs HDL

Schematic

- ✓ Good for multiple data flow
- ✓ Give overview picture
- ✓ Relate directly to hardware
- ✓ Don't need good programming skills
- ✓ High information density
- ✓ Easy back annotations
- ✓ Useful for mixed analogue/digital
- ✗ Not good for algorithms
- ✗ Not good for datapaths
- ✗ Poor interface to optimiser
- ✗ Poor interface to synthesis software
- ✗ Difficult to reuse
- ✗ Difficult to parameterise

HDL

- ✓ Flexible & parameterisable
- ✓ Excellent input to optimisation & synthesis
- ✓ Direct mapping to algorithms
- ✓ Excellent for datapaths
- ✓ Easy to handle electronically (only needing a text editor)
- ✗ Serial representation
- ✗ May not show overall picture
- ✗ Need good programming skills
- ✗ Divorce from physical hardware

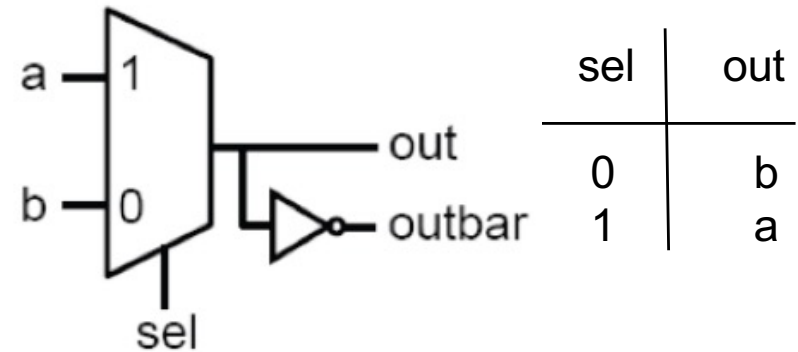
Verilog HDL

- ◆ Similar to C language to describe/specify hardware
- ◆ Description can be at different levels:
 - **Behavioural level**
 - **Register-Transfer Level (RTL)**
 - **Gate Level**
- ◆ Not only a specification language, also with associated **simulation environment**
- ◆ Easier to learn and “lighter weight” than its competition: VHDL
- ◆ Very popular with chip designers

- ◆ For this lecture, we will:
 - ❑ Learn through examples and practical exercises
 - ❑ Use two examples: 2-to-1 multiplexer and 7 segment decoder

Structure of a Module

- ◆ Verilog design contains **interconnected modules**
- ◆ A module has collections of low-level gates, statements, and other modules
- ◆ Here is an example of a simple module that describes a 2-to-1 multiplexer:



```
// Function: 2-to-1 multiplexer
module mux2to1 (out, outbar,
               a, b, sel);
```

```
    output out, outbar;
    input  a, b, sel;
```

```
    assign out = sel ? a : b;
    assign outbar = ~out;
```

```
endmodule
```

- ◆ // to end-of-line is comment. Can also use /* ... */ for multiline comments
- ◆ Declare and name module; list its ports; terminate with ‘;’

- ◆ Specify port as input, output (or inout if bidirectional)

- ◆ Express modules behaviour; each statement executes in parallel; **ORDER DOES NOT MATTER**

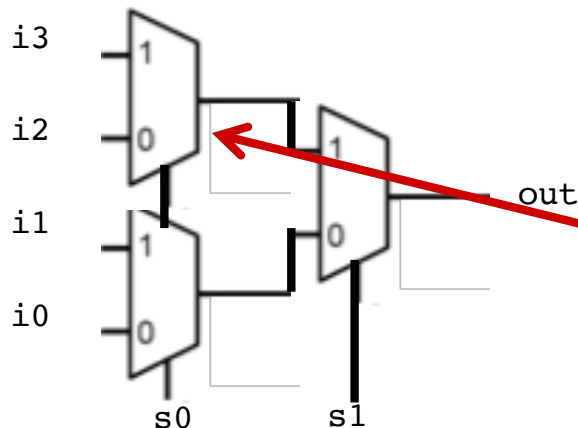
Continuous Assignment

```
module mux2to1 (out, outbar,  
               a, b, sel);
```

```
    output out, outbar;  
    input  a, b, sel;
```

```
    assign out = sel ? a : b;  
    assign outbar = ~out;
```

```
endmodule
```



- ◆ Keyword **assign** specifies **continuous assignment** to describe combinational logic
- ◆ Right-hand expression continuously evaluated responding to input changes immediately
- ◆ Left-hand is a net driven with evaluated value
- ◆ Left side must be a scalar, a net or a concatenation of nets and vector nets. (nets, vectors etc. will be described later)
- ◆ All continuous assignments execute in parallel
- ◆ Operators in expressions are low-level:
 - Conditional assignment: (cond)? vTrue: vFalse
 - Boolean: ~, &, |,
 - Arithmetic: +, -, *
- ◆ Operators can be nested. For example, here is a 4-to-1 mux:

```
assign out = s1 ? (s0 ? i3 : i2) : (s0 ? i1 : i0);
```

Description at Gate Level

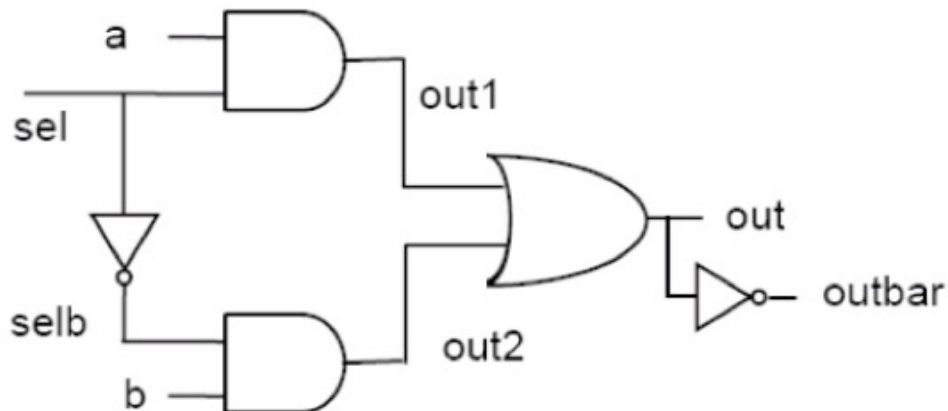
```
// Function: 2-to-1 multiplexer as gates
```

```
module mux_gate (out, outbar,  
                 a, b, sel);
```

```
    output  out, outbar;  
    input   a, b, sel;  
    wire    out1, out2, selb;
```

```
    not i1 (selb, sel);  
    and a1 (out1, a, sel);  
    and a2 (out2, b, selb);  
    or o1 (out, out1, out2);  
    not i2 (outbar, out);
```

```
endmodule
```



- ◆ Built-in logic gate primitives:
 - and, nand, or, nor, xor, xnor, not, buf
- ◆ Can use arbitrary number of inputs, e.g. and gate_name (out, in1, in2, in3, ...)
- ◆ Tri-state buffers: bufif1 and bufif0
- ◆ Connect gates with nets using declaration keyword wire
- ◆ **and a1 (out1, a, sel);**
- ◆ This statement is an **instantiation** of an AND gate. **a1** is the name of THIS particular AND gate.

Procedural Assignment using “always”

- ◆ Keyword **always** and **initial** are used to define **procedural assignment**, similar to procedures in software
- ◆ Good for behavioural description of hardware including combinational and sequential logic
- ◆ Support richer C-like constructs such as **if, for, while, case** etc.

```
module mux_2_to_1 (out, outbar, a, b, sel);  
    output out, outbar;  
    input a, b, sel;  
  
    reg out, outbar;  
  
    always @ (a or b or sel)  
  
    begin  
        if (sel) out = a;  
        else out = b;  
  
        outbar = ~out;  
    end  
endmodule
```

always @ (*)

- ◆ Declarations same as before
- ◆ Assignment inside an always block must be declared as variable data type such as **reg** (see later)
- ◆ **always** block runs once whenever a signal in the **sensitivity list** changes value
- ◆ Statements inside the always block are **executed sequentially**. ORDER MATTERS!
- ◆ Sandwiched between begin/end

Verilog “register” is NOT what it appears!

- ◆ Registers normally represents storage elements in digital logic, they need clock signals to update their output value
- ◆ In Verilog **reg** is NOT the same as a digital register, it is used only to declare a variable that holds a value
- ◆ Values of variables (declared as **reg**) can be changed anytime in a simulation, and can be used for nets of a combinational circuit
- ◆ In other words, in Verilog, **reg** is similar to declarations such as **int**, **real** etc. When you use:

reg signal_a;

whether a physical register (or D-FF) is synthesized to store **signal_a** or not would depend on the rest of the Verilog code.

case statement – better alternative to if-else

- ◆ **case** statement can often replace **if-else** construct within an **always** block, and provides better abstraction
- ◆ Here is an example using the mux_2_to_1 module:

```
always @ (a or b or sel)  
begin
```

```
    case (sel)  
        1'b0: out = b;  
        1'b1: out = a;  
    endcase
```

```
end
```

```
if (sel) out = a;  
else out = b;
```



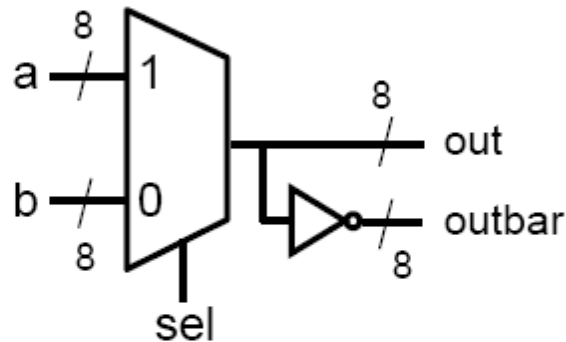
Notation for numbers:

<size> ' <base> <number>

2'b10	2 bit binary, v=2
'b10	Unsize binary 32-bit, v=2
31	Unsize decimal, v=31
8'hAf	8-bit hex, v=175
-16'd47	16-bit negative decimal, v=-47

n-bit signals - buses

- ◆ Verilog is powerful in specifying multi-bit signals and buses.
- ◆ Here is an example for 8-bit wide 2-to-1 multiplexer:

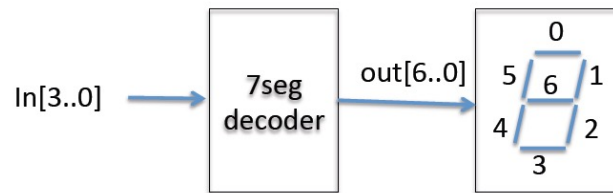


```
module mux_2_to_1(a, b, out,
                  outbar, sel);
    input [7:0] a, b;
    input sel;
    output [7:0] out, outbar;
    reg [7:0] out;
    always @ (a or b or sel)
    begin
        if (sel) out = a;
        else out = b;
    end
    assign outbar = ~out;
endmodule
```

Concatenate signals using the **{ }** operator

```
assign {b[7:0],b[15:8]} = {a[15:8],a[7:0]};
effects a byte swap
```

Putting everything together – 7 seg decoder



$In3 : in2$

out6	00	01	11	10
00	1	0	1	0
01	1	0	0	0
11	0	1	0	0
10	0	0	0	0

$In1 : in0$

$in[3..0]$	$out[6:0]$	Digit	$in[3..0]$	$out[6:0]$	Digit
0000	1000000	0	1000	0000000	8
0001	1111001	1	1001	0010000	9
0010	0100100	2	1010	0001000	A
0011	0110000	3	1011	0000011	b
0100	0011001	4	1100	1000110	C
0101	0010010	5	1101	0100001	d
0110	0000010	6	1110	0000110	E
0111	1111000	7	1111	0001110	F

$$out6 = /in3*/in2*/in1 + in3*in2*/in1*/in0 + /in3*in2*in1*in0$$

$$out5 = /in3*/in2*in0 + /in3*/in2*in1 + /in3*in1*in0 + in3*in2*/in1*in0$$

$$out4 = /in3*in0 + /in3*in2*/in1 + in3*/in2*/in1*in0$$

$$out3 = /in3*in2*/in1*/in0 + /in3*/in2*/in1*in0 + in2*in1*in0 + /in2*in1*/in0$$

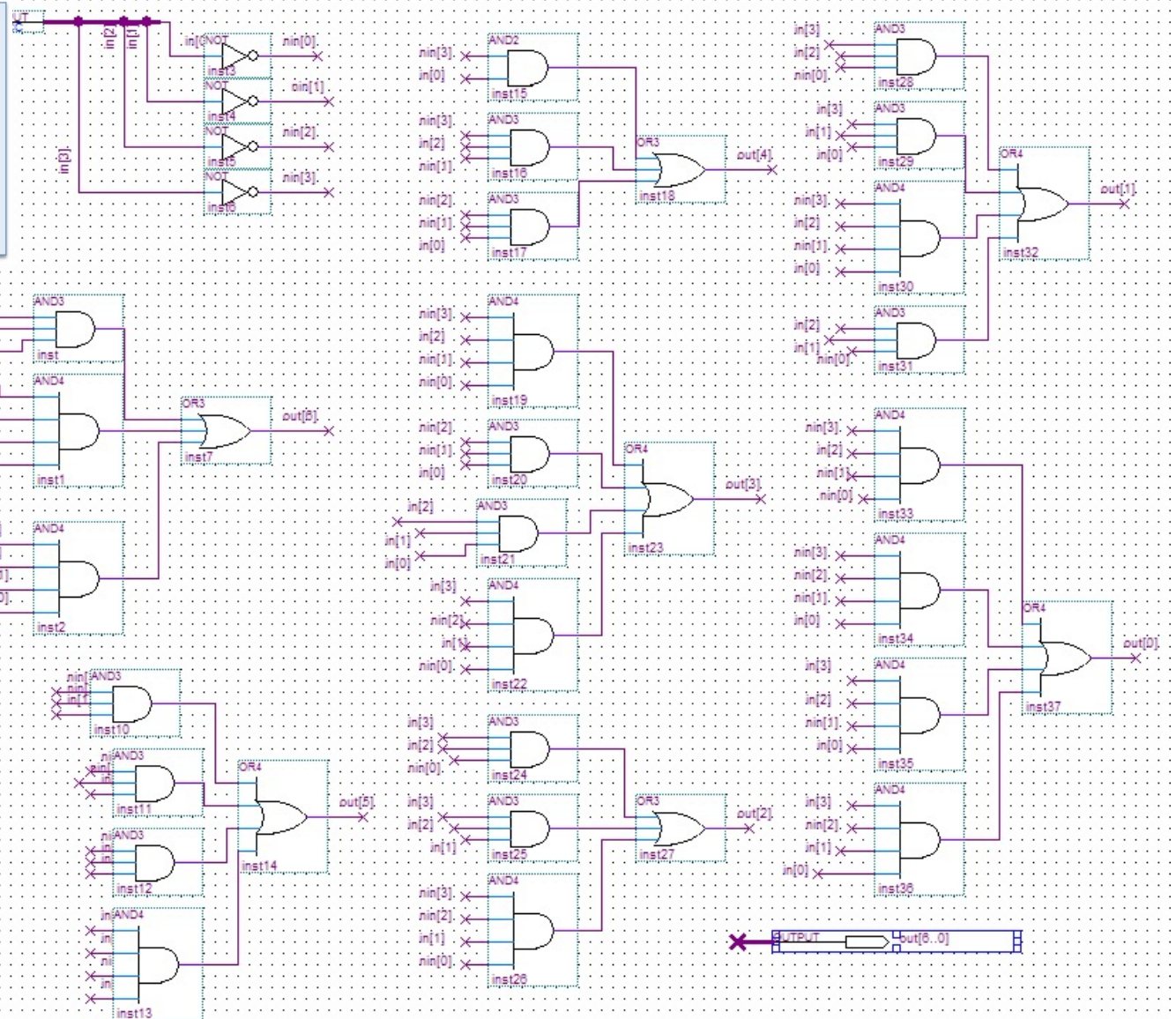
$$out2 = /in3*/in2*in1*/in0 + in3*in2*/in0 + in3*in2*in1$$

$$out1 = in3*in2*/in0 + /in3*in2*/in1*in0 + in3*in1*in0 + in2*in1*/in0$$

$$out0 = /in3*/in2*/in1*in0 + /in3*in2*/in1*/in0 + in3*in2*/in1*in0 + in3*/in2*in1*in0$$

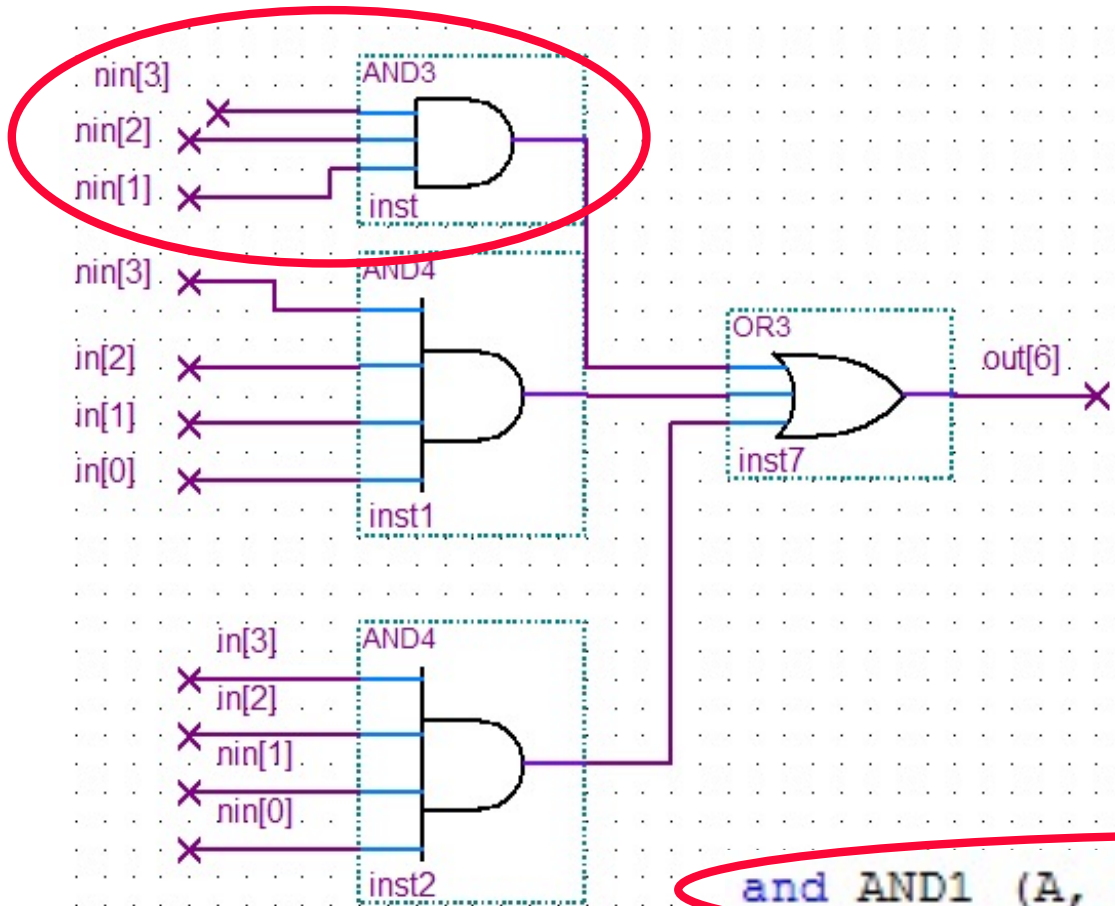
Method 1: Schematic Entry Implementation

$out6 = /in3*/in2*/in1 + in3*in2*/in1*/in0 + /in3*in2*in1*in0$
 $out5 = /in3*/in2*in0 + /in3*/in2*in1 + /in3*in1*in0 + in3*in2*/in1*in0$
 $out4 = /in3*in0 + /in3*in2*/in1 + in3*/in2*/in1*in0$
 $out3 = /in3*in2*/in1*/in0 + /in3*/in2*/in1*in0 + in2*in1*in0 + /in2*in1*/in0$
 $out2 = /in3*/in2*in1*/in0 + in3*in2*/in0 + in3*in2*in1$
 $out1 = in3*in2*/in0 + /in3*in2*/in1*in0 + in3*in1*in0 + in2*in1*/in0$
 $out0 = /in3*/in2*/in1*in0 + /in3*in2*/in1*/in0 + in3*in2*/in1*in0 + in3*/in2*in1*in0$



TEDIOUS!!!!

Method 2: Use primitive gates in Verilog



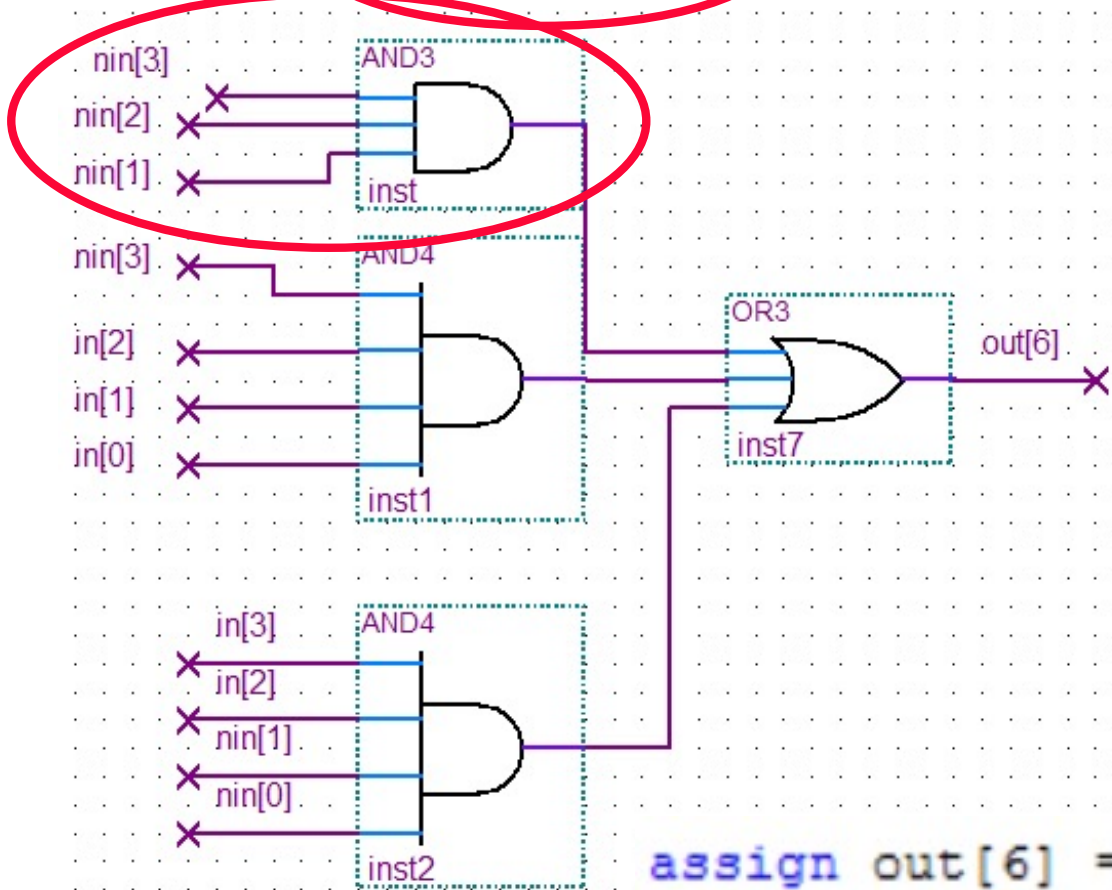
Equally TEDIOUS!!!!

Direct mapping of gates to primitives

```
and AND1 (A, nin[3], nin[2], nin[1]);  
and AND2 (B, nin[3], in[2], in[1], in[0]);  
and AND3 (C, in[3], in[2], nin[1], nin[0]);  
or OR1 (out[6], A, B, C);
```

Method 3: Use continuous assignment in Verilog

$$\text{out6} = \sim \text{in3} \cdot \sim \text{in2} \cdot \sim \text{in1} + \text{in3} \cdot \text{in2} \cdot \sim \text{in1} \cdot \sim \text{in0} + \sim \text{in3} \cdot \text{in2} \cdot \text{in1} \cdot \text{in0}$$



Much Better?

Direct mapping of Boolean equation using continuous assignment..

```
assign out[6] = ~in[3]&~in[2]&~in[1] |  
in[3]&in[2]&in[1]&~in[0] |  
~in[3]&in[2]&in[1]&in[0];
```

module & **endmodule**
sandwich the content of
this hardware module

Hex_to_7seg.v

```
//-----  
// Module name: hex_to_7seg  
// Function: convert 4-bit hex value to drive 7 segment display  
//           output is low active  
// Creator:   Peter Cheung  
// Version:   1.0  
// Date:      22 Oct 2011  
//-----
```

good header helps
documenting your code

specify interface to this
module as viewed from
outside

specify a 7-bit output bus,
out[6] ... out[0]

```
module hex_to_7seg (out,in);
```

declaration of
input and output
ports

```
    output [6:0] out;    // low-active output to drive 7 segment display  
    input  [3:0] in;     // 4-bit binary input of a hexademical number
```

```
    assign out[6] = ~in[3]&~in[2]&~in[1] | in[3]&in[2]&~in[1]&~in[0] |  
                   ~in[3]&in[2]&in[1]&in[0];  
    assign out[5] = ~in[3]&~in[2]&in[0] | ~in[3]&~in[2]&in[1] |  
                   ~in[3]&in[1]&in[0] | in[3]&in[2]&~in[1]&in[0];  
    assign out[4] = ~in[3]&in[0] | ~in[3]&in[2]&~in[1] | in[3]&~in[2]&~in[1]&in[0];  
    assign out[3] = ~in[3]&in[2]&~in[1]&~in[0] | ~in[3]&~in[2]&~in[1]&in[0] |  
                   in[2]&in[1]&in[0] | ~in[2]&in[1]&~in[0];  
    assign out[2] = ~in[3]&~in[2]&in[1]&~in[0] | in[3]&in[2]&~in[0] |  
                   in[3]&in[2]&in[1];  
    assign out[1] = in[3]&in[2]&~in[0] | ~in[3]&in[2]&~in[1]&in[0] |  
                   in[3]&in[1]&in[0] | in[2]&in[1]&~in[0];  
    assign out[0] = ~in[3]&~in[2]&~in[1]&in[0] | ~in[3]&in[2]&~in[1]&~in[0] |  
                   in[3]&in[2]&~in[1]&in[0] | in[3]&~in[2]&in[1]&in[0];
```

assign used to specify
combinational circuit

```
endmodule
```

Method 4: Power of behavioural abstraction

```

module hex_to_7seg (out,in);

    output [6:0] out;    // low-active output to
    input  [3:0] in;     // 4-bit binary input o

    reg [6:0] out;      // make out a variable

    always @ (in)
        case (in)
            4'h0: out = 7'b1000000;
            4'h1: out = 7'b1111001;    // -- 0 ---
            4'h2: out = 7'b0100100;    // |       |
            4'h3: out = 7'b0110000;    // 5       1
            4'h4: out = 7'b0011001;    // |       |
            4'h5: out = 7'b0010010;    // -- 6 ---
            4'h6: out = 7'b0000010;    // |       |
            4'h7: out = 7'b1111000;    // 4       2
            4'h8: out = 7'b0000000;    // |       |
            4'h9: out = 7'b0011000;    // -- 3 ---
            4'ha: out = 7'b0001000;
            4'hb: out = 7'b0000011;
            4'hc: out = 7'b1000110;
            4'hd: out = 7'b0100001;
            4'he: out = 7'b0000110;
            4'hf: out = 7'b0001110;
        endcase
    endmodule

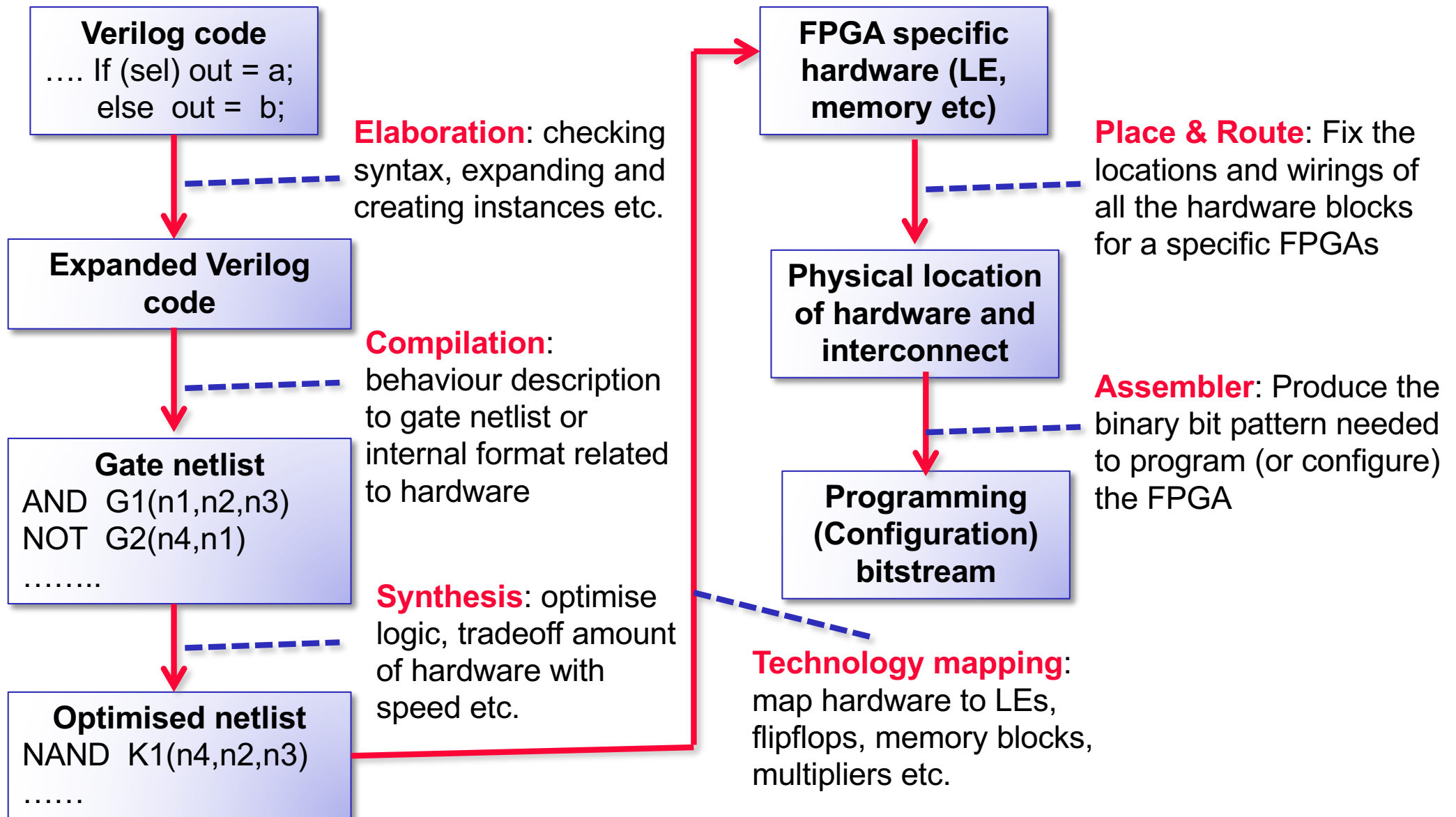
```

BEAUTIFUL !!!

- Direct mapping of truth table to case statement
- Close to specification, not implementation

in[3..0]	out[6:0]	Digit
0000	1000000	0
0001	1111001	1
0010	0100100	2
0011	0110000	3
0100	0011001	4
0101	0010010	5
0110	0000010	6
0111	1111000	7
1000	0000000	8
1001	0010000	9
1010	0001000	A
1011	0000011	b
1100	1000110	C
1101	0100001	d
1110	0000110	E
1111	0001110	F

From Verilog code to FPGA hardware



Power of Verilog: Integer Arithmetic

- ◆ Arithmetic operations make computation easy:

```
module add32(a, b, sum);  
    input[31:0] a,b;  
    output[31:0] sum;  
    assign sum = a + b;  
endmodule
```

- ◆ Here is a 32-bit adder with carry-in and carry-out:

```
module add32_carry(a, b, cin, sum, cout);  
    input[31:0] a,b;  
    input cin;  
    output[31:0] sum;  
    output cout;  
    assign {cout, sum} = a + b + cin;  
endmodule
```

Different types of Boolean Operators

- ◆ **Bitwise operators:** perform bit-sliced operations on vectors bit by bit
 - $\sim(4'b0101) = \{\sim 0, \sim 1, \sim 0, \sim 1\} = 4'b1010$
 - $4'b0101 \& 4'b0011 = 4'b0001$
- ◆ **Logical operators:** return true or false (1-bit) results
 - $!(4'b0101) = \sim 1 = 1'b0$
- ◆ **Reduction operators:** act on each bit of a SINGLE input vector
 - $\&(4'b0101) = 0 \& 1 \& 0 \& 1 = 1'b0$

Bitwise

$\sim a$	NOT
$a \& b$	AND
$a b$	OR
$a \wedge b$	XOR
$a \sim \wedge b$	XNOR

Logical

$!a$	NOT
$a \&\& b$	AND
$a b$	OR

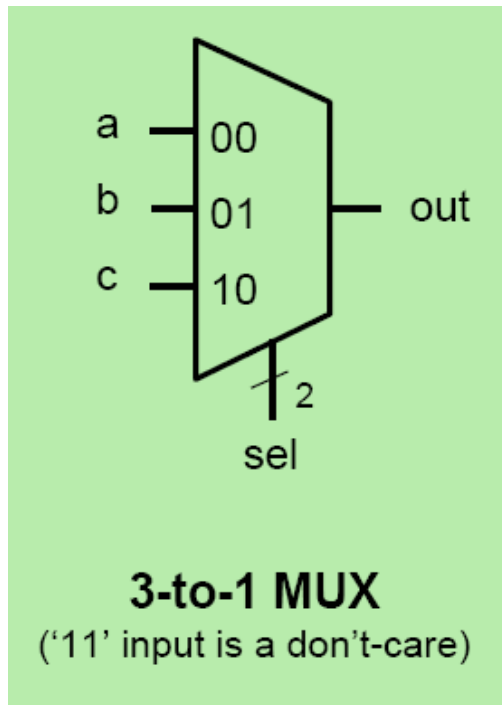
Note distinction between $\sim a$ and $!a$

Reduction

$\&a$	AND
$\sim \&$	NAND
$ $	OR
$\sim $	NOR
$\wedge$	XOR

Beware of Incomplete Specification

Intention



What you may write, but wrong!

```
module maybe_mux_3to1(a, b, c,
                      sel, out);

    input [1:0] sel;
    input a,b,c;
    output out;
    reg out;

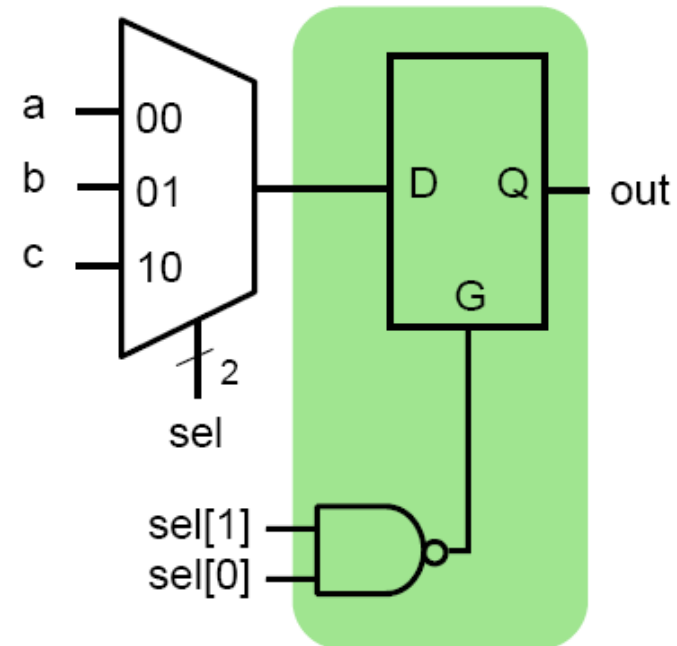
    always @(a or b or c or sel)
    begin
        case (sel)
            2'b00: out = a;
            2'b01: out = b;
            2'b10: out = c;
        endcase
    end
endmodule
```

Incomplete specification: adds unwanted latch circuit

if out is not assigned
during any pass through
the always block, then **the
previous value must be
retained!**

```
module maybe_mux_3to1(a, b, c,  
                      sel, out);  
  
    input [1:0] sel;  
    input a,b,c;  
    output out;  
    reg out;  
  
    always @(a or b or c or sel)  
    begin  
        case (sel)  
            2'b00: out = a;  
            2'b01: out = b;  
            2'b10: out = c;  
        endcase  
    end  
endmodule
```

Synthesized Circuit



- ◆ When sel = 2'b11, G = 0, therefore the latch stores the previous output value as required by Verilog in this situation.

Always avoid incomplete specification

- ◆ Solution 1: Precede all conditionals with a default assignment for all signals:

```
always @(a or b or c or sel)
begin
    out = 1'bx;
    case (sel)
        2'b00: out = a;
        2'b01: out = b;
        2'b10: out = c;
    endcase
end
endmodule
```

- ◆ Solution 2: Fully specify all branches of if-else construct, or include a default statement in case construct:

```
always @(a or b or c or sel)
begin
    case (sel)
        2'b00: out = a;
        2'b01: out = b;
        2'b10: out = c;
        default: out = 1'bx;
    endcase
end
endmodule
```

How to specify a sequential circuit?

- ◆ Edge-triggered flipflop is specified with: **always @ (posedge clk):**

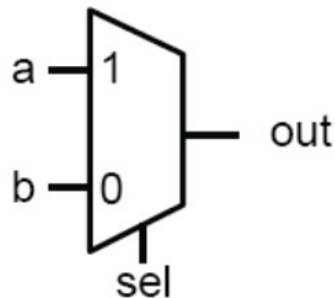
Combinational cct

```
module combinational(a, b, sel,
                    out);

    input a, b;
    input sel;
    output out;
    reg out;

    always @ (a or b or sel)
    begin
        if (sel) out = a;
        else out = b;
    end

endmodule
```



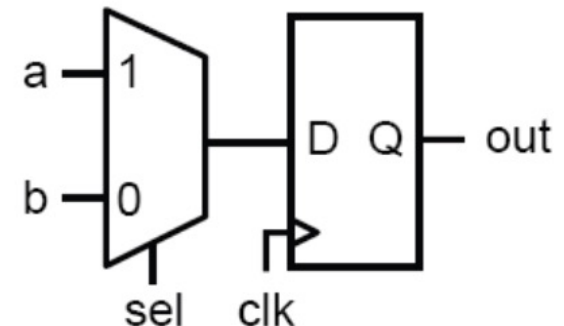
Sequential cct

```
module sequential(a, b, sel,
                 clk, out);

    input a, b;
    input sel, clk;
    output out;
    reg out;

    always @ (posedge clk)
    begin
        if (sel) out <= a;
        else out <= b;
    end

endmodule
```



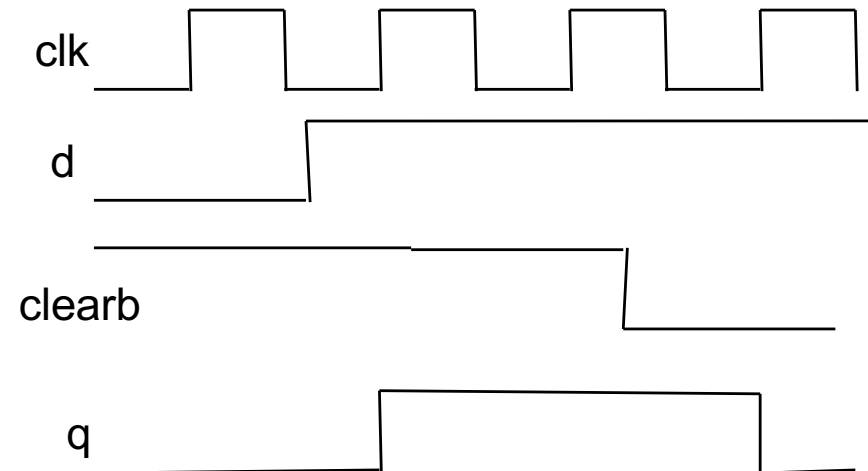
Synchronous clear in D-flipflop

- ◆ **posedge** and **negedge** makes an always block sequential and edge-triggered
- ◆ Sensitivity list in a sequential **always** block determines what circuit is synthesized

D flipflop with **synchronous** clear

```
module dff_sync_clear(d, clearb, clock, q);  
    input d, clearb, clock;  
    output q;  
    reg q;  
    always @ (posedge clock)  
    begin  
        if (!clearb) q <= 1'b0;  
        else q <= d;  
    end  
endmodule
```

+ve edge on clock triggers action in always block



- ◆ Beware of **race condition** if you have two or more always blocks – they execute in parallel!

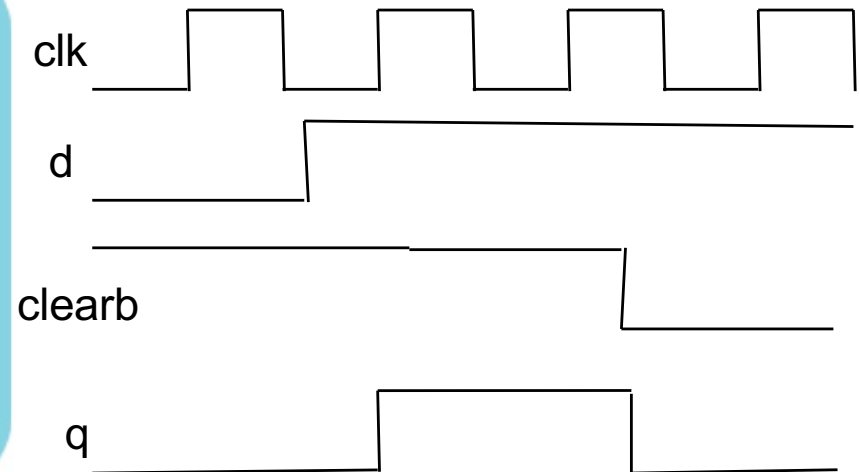
Asynchronous clear in D-flipflop

- ◆ If one signal in the sensitivity list uses **posedge** or **negedge**, ALL signals must also specify an edge. This syntax is wrong: **always @ (clearb or posedge clock)**

D flipflop with **asynchronous** clear

```
module dff_async_clear(d, clearb, clock, q);  
input d, clearb, clock;  
output q;  
reg q;  
  
always @ (negedge clearb or posedge clock)  
begin  
    if (!clearb) q <= 1'b0;  
    else q <= d;  
end  
endmodule
```

+ve edge on clock OR clearb trigger action in
always block



Blocking vs Non-blocking Assignments

- ◆ Verilog has two different types of assignments: **blocking** & **nonblocking**.
- ◆ Blocking assignments **=** are executed in the order they appear, therefore they are done one after another. Therefore the first statement “blocks” the second until it is done, hence it is called blocking assignments.

```
a = b;  
b = a;  
// both a & b = b
```

blocking

```
always @ (a or b or c)  
begin  
    x = a | b;  
    y = a ^ b ^ c;  
    z = b & ~c;  
end
```

blocking

x = a b;	1. Evaluate $a b$, assign result to x
y = a ^ b ^ c;	2. Evaluate a^b^c , assign result to y
z = b & ~c;	3. Evaluate $b \& (\sim c)$, assign result to z

- ◆ Non-blocking assignments **<=** are executed in parallel. Therefore an earlier statement does not block the later statement. Note the subtle effect this has within **always** block:

```
a <= b;  
b <= a;  
// swap a and b
```

Non-blocking

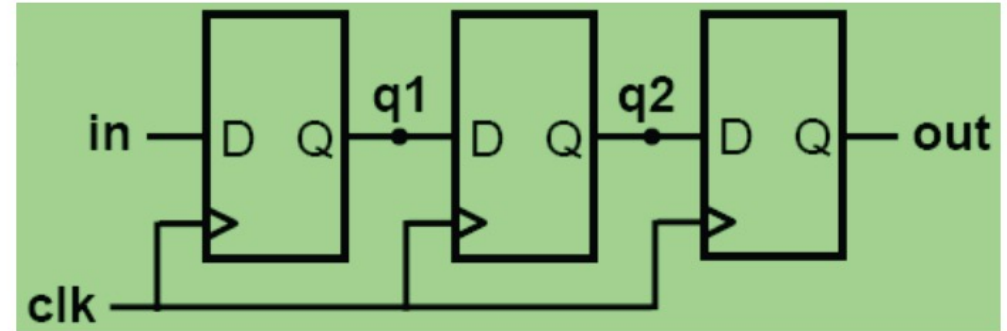
```
always @ (a or b or c)  
begin  
    x <= a | b;  
    y <= a ^ b ^ c;  
    z <= b & ~c;  
end
```

Non-blocking

x <= a b;	1. Evaluate $a b$ but defer assignment of x
y <= a ^ b ^ c;	2. Evaluate a^b^c but defer assignment of y
z <= b & ~c;	3. Evaluate $b \& (\sim c)$ but defer assignment of z
4. Assign x, y, and z with their new values	

Be careful to use the correct assignment

- ◆ Here are two versions of a 3-stage shift register consisting of 3 flipflops using blocking and nonblocking assignments.
- ◆ Will they give the same results?



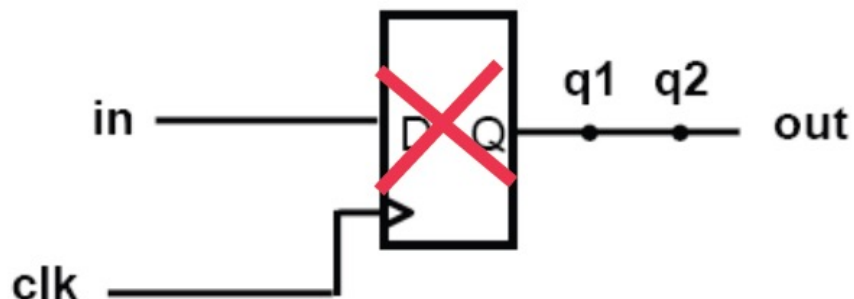
```
module blocking(in, clk, out);  
  input in, clk;  
  output out;  
  reg q1, q2, out;  
  always @ (posedge clk)  
  begin  
    q1 = in;  
    q2 = q1;  
    out = q2;  
  end  
endmodule
```

```
module nonblocking(in, clk, out);  
  input in, clk;  
  output out;  
  reg q1, q2, out;  
  always @ (posedge clk)  
  begin  
    q1 <= in;  
    q2 <= q1;  
    out <= q2;  
  end  
endmodule
```

Use NONBLOCKING assignment for sequential logic

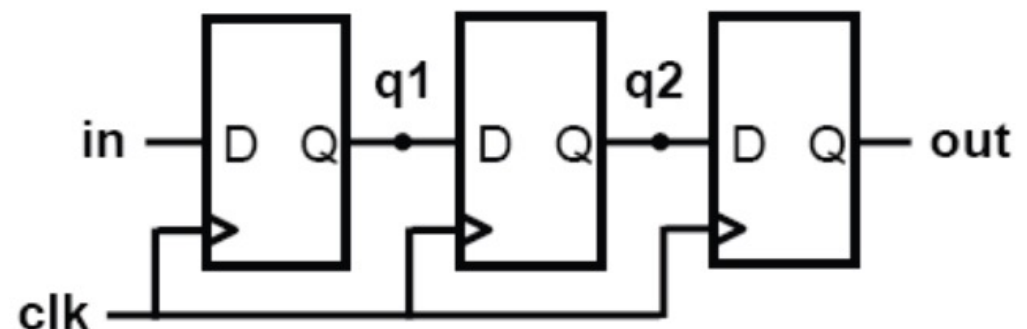
```
always @ (posedge clk)
begin
    q1 = in;
    q2 = q1;
    out = q2;
end
```

- ◆ At each rising clock edge:
 $q1 = in$,
then $q2 = q1 = in$,
then, $out = q2 = q1 = in$.
- ◆ Therefore $out = in$, which is NOT the intention.



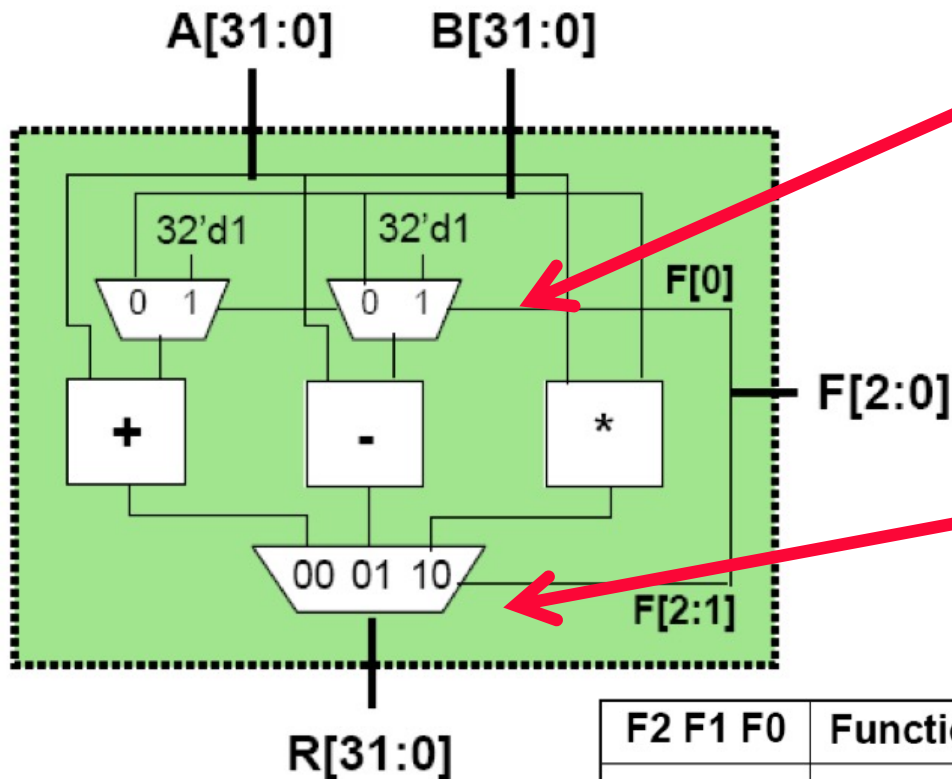
```
always @ (posedge clk)
begin
    q1 <= in;
    q2 <= q1;
    out <= q2;
end
```

- ◆ At each rising clock edge, $q1$, $q2$ and out simultaneously receive the old values of in , $q1$, $q2$ respectively.



A larger example – 32-bit ALU in Verilog

- Here is an 32-bit ALU with 5 simple instructions:



F2	F1	F0	Function
0	0	0	A + B
0	0	1	A + 1
0	1	0	A - B
0	1	1	A - 1
1	0	X	A * B

2-to-1 MUX

```

module mux32two(i0,i1,sel,out);
input [31:0] i0,i1;
input sel;
output [31:0] out;

assign out = sel ? i1 : i0;

endmodule
    
```

3-to-1 MUX

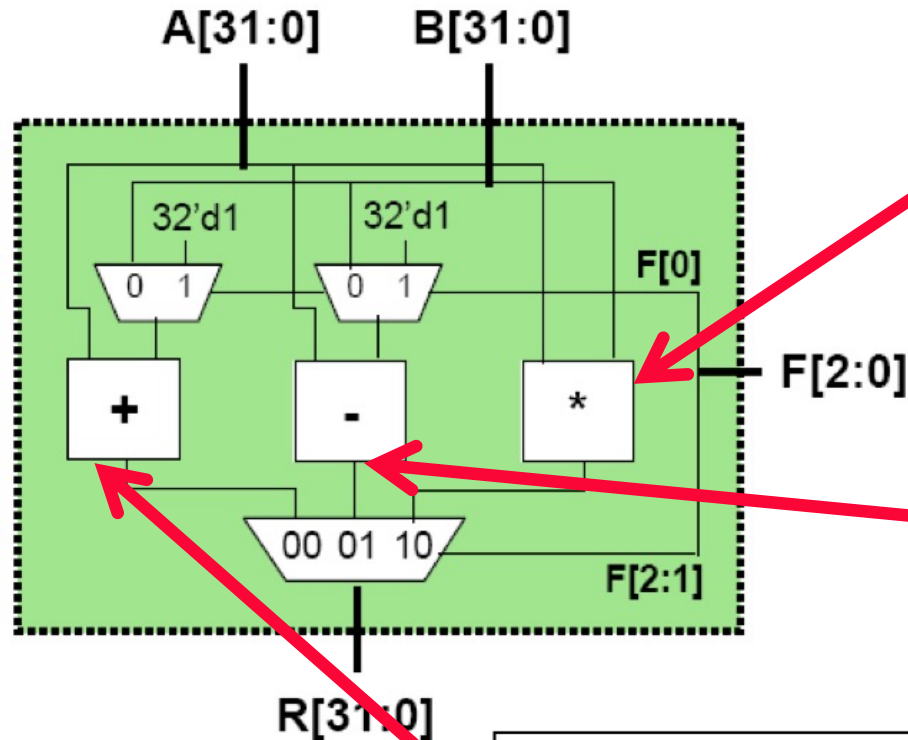
```

module mux32three(i0,i1,i2,sel,out);
input [31:0] i0,i1,i2;
input [1:0] sel;
output [31:0] out;
reg [31:0] out;

always @ (i0 or i1 or i2 or sel)
begin
    case (sel)
        2'b00: out = i0;
        2'b01: out = i1;
        2'b10: out = i2;
        default: out = 32'bx;
    endcase
end
endmodule
    
```

The arithmetic modules

- ◆ Here is an 32-bit ALU with 5 simple instructions:



```
module mul16(i0,i1,prod);
input  [15:0] i0,i1;
output [31:0] prod;

// this is a magnitude multiplier
// signed arithmetic later
assign prod = i0 * i1;

endmodule
```

```
module sub32(i0,i1,diff);
input  [31:0] i0,i1;
output [31:0] diff;

assign diff = i0 - i1;

endmodule
```

```
module add32(i0,i1,sum);
input [31:0] i0,i1;
output [31:0] sum;

assign sum = i0 + i1;

endmodule
```

Top-level module – putting them together

- Given submodules:

```
module mux32two(i0,i1,sel,out);
module mux32three(i0,i1,i2,sel,out);
module add32(i0,i1,sum);
module sub32(i0,i1,diff);
module mul16(i0,i1,prod);
```

```
module alu(a, b, f, r);
  input [31:0] a, b;
  input [2:0] f;
  output [31:0] r;
```

```
  wire [31:0] addmux_out, submux_out;
  wire [31:0] add_out, sub_out, mul_out;
```

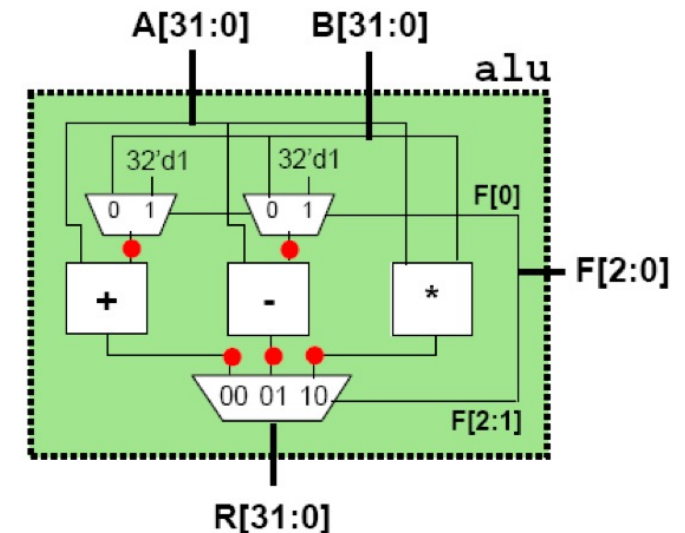
```
  mux32two    adder_mux(b, 32'd1, f[0], addmux_out);
  mux32two    sub_mux(b, 32'd1, f[0], submux_out);
  add32       our_adder(a, addmux_out, add_out);
  sub32       our_subtractor(a, submux_out, sub_out);
  mul16       our_multiplier(a[15:0], b[15:0], mul_out);
  mux32three  output_mux(add_out, sub_out, mul_out, f[2:1], r);
```

endmodule

module
names

(unique)
instance
names

corresponding
wires/regs in
module alu



intermediate output nodes ●